

5/29

<Function & AI>

- 함수: 입력값과 출력값이 있는 구조
Input → (function) → Output
⇒ AI도 결국 입력값을 받아서 함수를 거친 뒤, 출력값을 만들어내는 구조
- STT: Speech to Text
말소리 → (AI function) → 텍스트
- TTS: Text to Speech
텍스트 → (AI function) → 음성
- Text → Text (ex. 챗지피티)
텍스트 → 텍스트
- LLM: Large Language Model
많은 양의 텍스트 데이터를 학습해서 언어를 이해하고, 생성.

<Data & Vectorization>

- Data
 - 숫자
 - Image, speech, text⇒ Data should be *vectorized*
- Vector: data visualization (데이터 시각화)
 - 길이보다는, 각도(cosine)로 거의 모든 것이 포함됨.
 - 벡터는 원래 한줄. * But 여러 줄일 경우, 한 줄로 길게 쭉 나열하면 됨.
 - AI에 벡터가 들어가서, 벡터가 나오는 구조.

➤ 이미지, 음성, 텍스트도 모두 데이터로 취급됨.

AI 입장에서서는 결국 이 모든 데이터가 숫자로 바뀌어야 처리가 가능!

즉, AI는 사진을 “사진”으로 보는 것이 아니라 숫자들의 배열로 보고, 문장도 단어의 의미를 숫자 벡터로 바꾸어 처리함.

<Matrix & Deep Learning>

- 하나의 행렬이 아닌 여러 행렬을 사용할 때:
입력에서 출력값까지의 과정이 더 멀어짐, ‘깊어짐’.
= **Deep learning** (말그대로 깊게 배운다는 뜻이 x, 행렬이 많이 쓰였다는 뜻!)
- Parameter: 매개 변수의 개수
 - AI 모델 내부에서 학습을 통해 조정되는 값
 - 챗지피티 3.0의 경우, 1750억 개 – 어마어마한 양의 전기를 사용

요약)

- Function: 입출력, 사람, 사람같은 AI, AI의 입출력
- Vector: data, a sequence of number, a point
- Matrix: AI function

➤ Deep learning에서 'deep'은 단순히 AI가 많이 안다거나 똑똑하다는 의미가 x. 입력값이 여러 행렬과 함수를 거치면서 점점 변환되고, 그 과정이 여러 층으로 쌓이기 때문에 deep learning이라고 불리는 것. 또한, parameter는 AI가 학습 과정에서 조정하는 값이기 때문에, AI의 성능과 깊게 관련됨.

<How AI Learns>

- 어떻게 좋은 행렬을 구할 수 있는가?
 - 입력을 넣고
 - 예측한 뒤
 - 제대로 예측할 때까지 반복 (제대로 '학습'을 할 때까지!)

➤ AI의 학습은 단순히 정보를 저장하는 과정이 x. 입력을 바탕으로 예측하고, 그 예측이 틀리면 parameter를 조정하면서 다시 학습함.

즉, AI가 '학습한다'는 것은 사람이 외우는 방식과는 다르게, 더 정확한 출력을 만들 수 있도록 내부의 값들을 계속 수정해 나가는 과정.

- AI chat system in call center
(사람 → STT → LLM → TTS → 사람)의 구조

Need...

- 정확성
- 빠른 속도
- 저렴한 비용

- What to unlearn and relearn?
 - 지금까지 배운 것은 다 틀렸다고 생각할 것.
 - AI 시대에는 기존 방식만 고집하기보다 새롭게 배우고 다시 익히는 태도가 필요.

AGI coming soon, and AI rules over human?

- AGI는 아직 없다.

➤ AI를 배울 때는 기술적인 내용뿐 아니라, AI를 바라보는 태도도 중요. AI가 빠르게 발전하고 있기에 지금까지 배운 지식이나 방식이 항상 정답이라고 생각하기보다는, 계속 새롭게 배우고 수정하는 태도가 필요하다고 느꼈음. 또한, AGI가 곧 인간을 지배할 것처럼 과장해서 받아들이기보다, 현재 AI가 무엇을 할 수 있고 무엇을 아직 할 수 없는지 구분해서 이해하는 것이 중요해보임.

- Solopreneur – 1인 기업; 나만의 사업을 만드는 것을 목표로 할 것!
- **팔란티어** (온톨로지, 지식그래프)) 사람이 가능한, 엄청나게 **많이** 개입해야 함 (아니면 어차피 망하니까. AI가 아무리 잘해도, 효율성을 극대화하기 위해!)
VS
- **프론티어** (엔트로픽)) 사람의 개입을 최대한 **줄여야** 함.
- **Agentic**: LLM에 loop를 걸어둔 기술 (한 번 해서는 안되니까)
만족/잘됐다 할 때까지 계속 돌아감 ; 중간 과정에서 자동적으로 correction이 이루어짐.
→ rule처럼 딱딱한 workflow에 비해 자율성을 더 많이 가짐.

Case 1 (팔란티어 방법): 미리 체계적으로 데이터 개체로 만들고 서로 연결해두어, '지식그래프'를 만들.

*지식그래프: 노드 – (edge) – 노드

*노드들을 연결해주는 것이 edge

Forward Development Engineer (FDE): 회사 사람들과 대화하며 노드, edge를 만들.

LLM) raw data를 받아, 오늘의 그래프를 자동적으로 만들어줌.

Since raw data 매일매일 달라짐 → 이를 기반으로 dynamic 그래프를 만드는 것을 자동화 시킴.

↳ 이 과정이 생략되어 있고, raw data만 넣으면 되는 방식이 **엔트로픽**

Case 2: 아무런 정보 없이 LLM 혼자 결정할 때 (극단적)

- 자신의 매개변수에 저장된 지식만을 바탕으로, 그럴듯한 답변을 생성함 (군사 교범만 학습시켜둠)
- 유지관리보수가 편함, 단순해서 보기도 쉬움.

➤ 앞으로는 AI 자체의 성능뿐 아니라 데이터를 어떻게 구조화하고 연결할 것인지, 그리고 사람이 어느 지점에서 개입해야 하는지가 매우 중요해질 것.